

**Национальный исследовательский технологический университет**  
**«МИСИС»**  
**(НИТУ МИСИС)**

Институт: Институт компьютерных наук (ИKN)

**ОТЧЁТ ПО ПРАКТИКЕ**

**«Разработка локальной информационно-  
поисковой RAG-системы PyAnswer по  
русскоязычной базе знаний Python»**

Выполнил студент:

**Горбачёв Вадим Александрович**

Группа: БИВТ-24-2

Руководитель практики:

Валова А. А.

Москва

2026

## Содержание

|                                                    |    |
|----------------------------------------------------|----|
| Содержание.....                                    | 2  |
| Введение.....                                      | 4  |
| 1. Анализ предметной области.....                  | 5  |
| 2. Анализ требований.....                          | 6  |
| 3. Цель и задачи работы.....                       | 7  |
| 4. Этапы выполнения проекта.....                   | 8  |
| 5. Архитектура системы.....                        | 9  |
| 6. Сбор данных.....                                | 10 |
| 7. Обработка данных.....                           | 12 |
| 8. Проектирование базы данных.....                 | 13 |
| 9. Примеры SQL-запросов.....                       | 15 |
| 10. Реализация поисковой системы.....              | 17 |
| 10.1. BM25.....                                    | 17 |
| 10.2. Vector Search.....                           | 17 |
| 10.3. Hybrid Search и reranker.....                | 17 |
| 11. Qdrant и управление индексом.....              | 19 |
| 12. Реализация RAG.....                            | 20 |
| 13. Пользовательский интерфейс.....                | 21 |
| 14. Модель авторизации.....                        | 24 |
| 15. Роли и права доступа.....                      | 25 |
| 16. API системы.....                               | 27 |
| 17. Фоновые задания.....                           | 28 |
| 18. Контейнеризация и запуск.....                  | 29 |
| 19. Тестирование.....                              | 30 |
| 20. Результаты работы.....                         | 31 |
| 21. Соответствие требованиям задания.....          | 32 |
| 22. Полнота фактической реализации.....            | 33 |
| 22.1. Сбор, возобновление и обработка корпуса..... | 33 |

|                                                              |    |
|--------------------------------------------------------------|----|
| 22.2. PostgreSQL, ER-модель и миграции.....                  | 35 |
| 22.3. Индексирование, три режима поиска и ранжирование.....  | 36 |
| 22.4. Локальный RAG и защита контекста.....                  | 37 |
| 22.5. React-интерфейс и пользовательские функции.....        | 38 |
| 22.6. Аутентификация, sessions и RBAC.....                   | 39 |
| 22.7. EDITOR, ADMIN и безопасный audit.....                  | 40 |
| 22.8. Контейнеризация, устойчивость и единый origin.....     | 41 |
| 22.9. Проверки, реальная работа и измеренные показатели..... | 42 |
| 22.10. SQL, документация и материалы защиты.....             | 43 |
| 22.11. Индивидуальный вклад автора.....                      | 44 |
| 22.12. Таблица полноты реализации.....                       | 44 |
| 23. Соответствие требованиям кейса «Поисковик, RAG».....     | 51 |
| 24. Ограничения и направления развития.....                  | 52 |
| Заключение.....                                              | 53 |
| Список источников.....                                       | 54 |
| Приложение А. Полная ER-диаграмма.....                       | 55 |
| Приложение Б. Полный SQL-файл.....                           | 56 |
| Приложение В. Карта исходного кода.....                      | 57 |
| Приложение Г. Материалы подтверждения.....                   | 58 |

## Введение

Объём технической информации по языку Python постоянно растёт: вопросы, ответы, примеры кода и обсуждения распределены по тысячам страниц. Обычный последовательный просмотр не позволяет быстро найти релевантное объяснение, особенно если пользователь формулирует задачу иначе, чем автор исходного материала. Информационно-поисковая система решает эту проблему за счёт индексирования корпуса, ранжирования документов и представления проверяемой ссылки на источник.

Подход Retrieval-Augmented Generation (RAG) дополняет поиск генерацией связного ответа на основе найденных фрагментов. В отличие от генерации без источников, RAG позволяет ограничить модель фактическим контекстом, показать цитаты и отказаться от ответа при недостаточной доказательной базе. Локальное выполнение embedding- и generation-моделей дополнительно исключает передачу содержимого корпуса во внешний облачный LLM API и делает конфигурацию воспроизводимой.

PyAnswer — локальная информационно-поисковая система по русскоязычным вопросам о Python. Система получает данные Stack Overflow на русском через Stack Exchange API, хранит первичный корпус в PostgreSQL, создаёт sparse/dense индекс в Qdrant, выполняет BM25, vector и hybrid retrieval, применяет локальный reranker и формирует RAG-ответ через Ollama с проверяемыми цитатами.

**Объект работы** — информационно-поисковые системы по техническим базам знаний. **Предмет работы** — методы сбора, очистки, индексирования, гибридного поиска, ранжирования и генерации ответов по найденным источникам. Отчёт соответствует фактическому состоянию commit 60cc0680cba0934d2359701778e743f51f397a8d и Alembic revision 20260716\_0006.

## 1. Анализ предметной области

Stack Overflow на русском содержит практические вопросы, ответы, фрагменты программного кода, оценки и теги. Эти данные неоднородны: HTML-разметка смешана с кодом; один вопрос имеет несколько ответов; формулировки пользователей различаются; записи обновляются; часть результатов может стать неактуальной. Поэтому проект должен решать одновременно задачи сбора, нормализации, хранения истории, полнотекстового и семантического поиска.

Единицей корпуса PyAnswer является документ, соответствующий вопросу. В него входят метаданные вопроса, очищенный текст, принятый и выбранные дополнительные ответы, теги, fenced code blocks, исходная ссылка, авторство и доступная лицензия источника. Такая гранулярность сохраняет смысл обсуждения и позволяет формировать несколько поисковых chunks без потери attribution.

PostgreSQL выбран источником истины: в нём находятся документы, пользователи, роли, задания, история изменений, аудит и сохранённые RAG-ответы. Qdrant хранит производное представление chunks и может быть полностью перестроен из PostgreSQL. Это разделение предотвращает превращение поискового индекса в единственное хранилище данных.

## 2. Анализ требований

Требования кейса «Поисковик, RAG» сопоставлены с исполняемыми модулями и проверочными артефактами в таблице 1.

**Таблица 1 — Реализация требований кейса**

| Требование                       | Реализация в PyAnswer                                           | Статус    |
|----------------------------------|-----------------------------------------------------------------|-----------|
| Парсинг информационного ресурса  | backend/app/integrations/stackexchange/, workers/source_sync.py | Выполнено |
| Загрузка в поисковые индексы     | services/search_index.py, workers/indexer.py                    | Выполнено |
| Сервер поисковых запросов        | FastAPI GET /api/search                                         | Выполнено |
| Выдача документов, top-N и score | Search API и SearchPage.tsx                                     | Выполнено |
| Содержание и ссылка на источник  | GET /api/documents/{id}, source_url                             | Выполнено |
| RAG/суммаризация                 | /api/ask, /api/ask/stream, services/rag.py                      | Выполнено |
| Hybrid Search                    | weighted RRF в search/fusion.py                                 | Выполнено |
| Reranking                        | локальный reranker service                                      | Выполнено |
| Фильтрация                       | search/filters.py, параметры Search API                         | Выполнено |
| UI                               | React/Vite frontend                                             | Выполнено |
| Авторизация и роли               | session auth, CSRF, USER/EDITOR/ADMIN                           | Выполнено |
| Аудит                            | audit_events, admin audit API/UI                                | Выполнено |
| Фоновые задания                  | durable PostgreSQL jobs, worker/indexer                         | Выполнено |

Обязательные требования закрыты. Расширенные функции реализованы без ослабления серверной модели доступа: скрывание элемента frontend не заменяет permission check в FastAPI.

### 3. Цель и задачи работы

**Цель работы** — разработать локальную информационно-поисковую RAG-систему по русскоязычной базе знаний Python.

Для достижения цели решены следующие задачи:

- выбран источник Stack Overflow на русском и разработан клиент Stack Exchange API;
- создано хранение документов, пользователей и операций в PostgreSQL;
- реализованы очистка HTML, сохранение кода, нормализация и exact-дедупликация;
- реализованы revisions и детерминированный чанкинг;
- построены sparse BM25 и dense HNSW индексы;
- реализованы BM25, Vector и Hybrid Search с weighted RRF;
- интегрирован локальный reranker;
- интегрированы локальные embedding- и generation-модели Ollama;
- реализован grounded RAG со streaming и цитатами;
- создан React-интерфейс;
- реализованы session authorization, CSRF и роли USER, EDITOR, ADMIN;
- реализованы durable jobs, lease, heartbeat, retry и cancellation;
- настроены Docker Compose, healthchecks, migrations, bootstrap и проверочные процедуры;
- выполнены backend/frontend tests, consistency и acceptance checks.

## 4. Этапы выполнения проекта

Фактическая структура migrations, документации и итогового commit подтверждает последовательное развитие проекта: анализ задания; архитектурное проектирование; frontend-прототип; FastAPI backend и PostgreSQL schema; authorization/RBAC; ingestion worker; очистка, дедупликация и chunking; Qdrant indexer; BM25/vector/hybrid retrieval; reranker; Ollama и RAG; тестирование; подготовка manifests, скриншотов и материалов защиты.

Alembic содержит шесть последовательных revisions: начальная схема, durable worker queue, processing/chunks, версии индекса, hybrid search и local RAG. Такое разбиение отражает развитие системы и обеспечивает воспроизводимое обновление базы до head 20260716\_0006.

## 5. Архитектура системы

Система состоит из восьми Docker Compose services: postgres, qdrant, ollama, backend, worker, indexer, reranker, frontend. Nginx входит в production image frontend и обеспечивает единый origin, статическую раздачу React и reverse проxy /api к FastAPI.

На рисунке 1 показаны два основных потока. Первый: Stack Exchange API → ingestion worker → PostgreSQL → indexer → Qdrant. Второй: Browser → Nginx/React → FastAPI → PostgreSQL/Qdrant → reranker/Ollama. Очередь заданий реализована в PostgreSQL; отдельный message broker отсутствует.

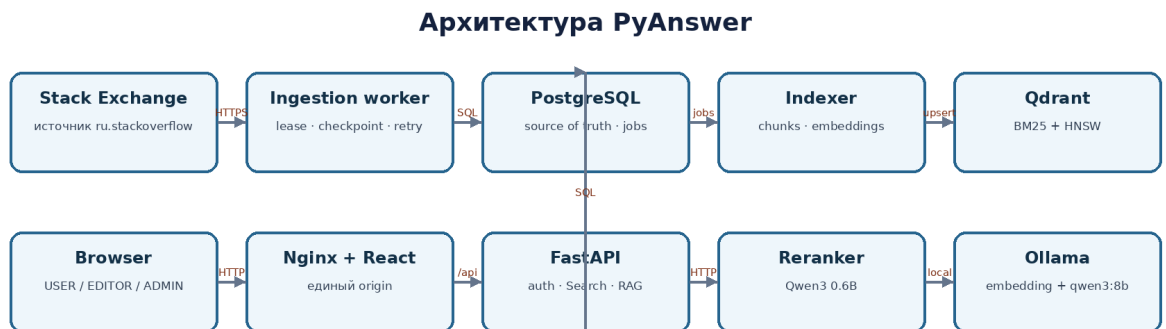


Рисунок 1 — Архитектура сервисов PyAnswer

Browser не обращается напрямую к PostgreSQL, Qdrant, Ollama или reranker. FastAPI является единственной серверной границей для authentication, authorization, search и RAG. PostgreSQL остаётся source of truth, а Qdrant — восстанавливаемым индексом.

## 6. Сбор данных

Источник корпуса — Stack Overflow на русском (`ru.stackoverflow`), тег `python`, API version 2.3. Клиент реализует pagination, page size, quota accounting, ограничение частоты запросов, backoff, обработку `Retry-After`, timeouts, лимит ответа и bounded retries. Полный импорт не стартует автоматически: uncapped sync защищён явным подтверждением.

Для массового первичного наполнения дополнительно реализован импорт официального Stack Exchange Data Dump. Модуль `backend/app/integrations/stackexchange/dump.py` потоково читает `Posts.xml` и `Users.xml` из архива `ru.stackoverflow.com.7z`, выбирает вопросы с тегом `python`, отдельным ограниченным проходом собирает полный набор ответов и присоединяет имена авторов. CLI `backend/app/scripts/import_stackexchange_dump.py` передаёт ветки в существующий `ingest_question_thread`, поэтому очистка HTML, сохранение кода, SHA-256, дедупликация, revisions и чанкинг не дублируются. Команда `make import-dump` требует явных `RUN_DATA_DUMP_IMPORT=1` и `DUMP_ARCHIVE`; предусмотрены `parse-only` проверка, JSON-отчёт и идемпотентное восстановление пропусков ответов. Реальный массовый импорт при формировании отчёта не запускался; реализация подтверждена unit-тестами `backend/tests/unit/test_stackexchange_dump.py`.

Ingestion worker получает durable job из PostgreSQL атомарным claim. Lease ограничивает время владения заданием, heartbeat продлевает его, checkpoint хранит page и item offset только после committed batch. После restart другой worker может продолжить работу. Cancellation проверяется между безопасными этапами; retry использует `attempt`, `max_attempts` и `next_attempt_at`.

Фактический corpus manifest содержит 10 720 документов, из которых 10 700 — реальные обработанные записи и 20 — demo. В корпусе 10 261 ответ, 10

165 выбранных ответов, 1 475 тегов, 25 214 chunks и 10 700 revisions. Initial sync формально не завершён: целевое значение 25 000 не достигнуто из-за квоты источника, однако acceptance criterion не менее 5 000 реальных обработанных документов выполнен.

## 7. Обработка данных

Конвейер обработки показан на рисунке 2. HTML разбирается parser-ом, потенциально опасные элементы удаляются, а программный код сохраняется в fenced blocks. Заголовки и URL нормализуются. Identity deduplication сопоставляет source/external ID; exact-content deduplication использует SHA-256 canonical text.



Рисунок 2 — Конвейер сбора и обработки документов

Изменение контента создаёт запись `document_revisions` с version, hashes и JSON snapshot. Chunks формируются детерминированно; ключ chunk связан с версией документа и ordinal. Ошибка одного thread фиксируется в `ingestion_failures` и не должна откатывать уже committed независимые документы. Фактический manifest показывает 0 незакрытых ingestion failures на момент формирования.

## 8. Проектирование базы данных

Фактическая PostgreSQL 16 schema после migration head содержит 29 таблиц, включая служебную `alembic_version`, и 48 внешних ключей. Основные области: authentication (`users`, `roles`, `permissions`, `sessions`); corpus (`sources`, `documents`, `answers`, `tags`, `chunks`, `revisions`); operations (`jobs`, `job_events`, `workers`, `failures`); search index versions/runs; RAG responses/sources; audit и settings.

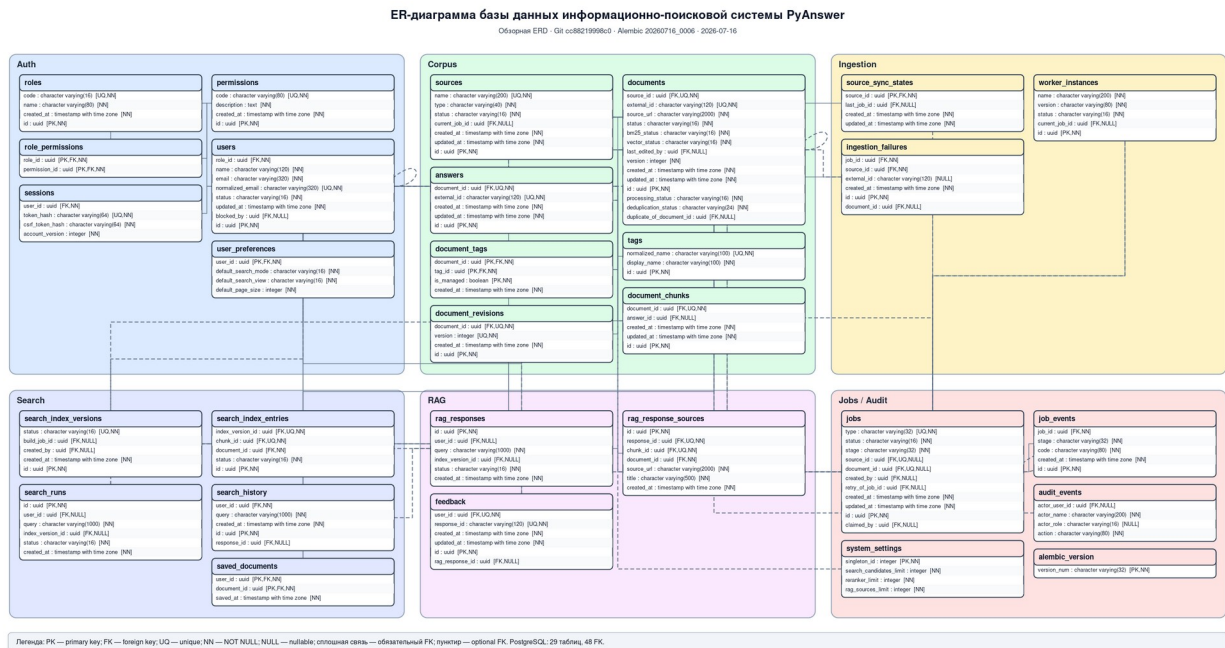


Рисунок 3 — Обзорная ER-диаграмма PostgreSQL PyAnswer

PK однозначно идентифицируют записи, FK обеспечивают ссылочную целостность. Примеры 1:1 — `users` → `user_preferences`; 1:N — `sources` → `documents`, `documents` → `answers/chunks/revisions`, `jobs` → `job_events`; N:M — `roles/permissions` через `role_permissions`, `documents/tags` через `document_tags`, `users/documents` через `saved_documents`.

`document_revisions` хранит историю контента; `job_events` — хронологию выполнения задания; `audit_events` — действия actor-a и sanitized before/after; `search_index_versions` — blue-green версии

Qdrant; rag\_response\_sources — соответствие citation index конкретным documents/chunks. Qdrant не входит в ERD, поскольку не является реляционным source of truth.

## 9. Примеры SQL-запросов

Ниже приведены сокращённые read-only запросы из docs/final/sql-demo.sql.

**JOIN — документы и источники.** Цель: показать происхождение документа.

```
SELECT d.external_id, d.normalized_title, s.name, d.source_url
FROM documents d JOIN sources s ON s.id = d.source_id
ORDER BY d.created_at DESC LIMIT 20;
```

Результат содержит идентификатор, заголовок, источник и URL; используются documents, sources.

**GROUP BY — документы по статусам.**

```
SELECT status, processing_status, count(*) AS documents
FROM documents GROUP BY status, processing_status
ORDER BY documents DESC;
```

Результат агрегирует moderation и processing state таблицы documents.

**Подзапрос — документы без актуальной index entry.**

```
SELECT d.external_id, d.normalized_title
FROM documents d
WHERE NOT EXISTS (
  SELECT 1 FROM search_index_entries e
  JOIN search_index_versions v ON v.id=e.index_version_id AND v.status='ACTIVE'
  WHERE e.document_id=d.id AND e.document_version=d.version AND e.status='INDEXED'
);
```

**CTE — последнее задание каждого источника.**

```
WITH ranked_jobs AS (
  SELECT j.*, row_number() OVER
    (PARTITION BY source_id ORDER BY created_at DESC, id DESC) AS rn
  FROM jobs j WHERE source_id IS NOT NULL
)
```

```
SELECT s.name, r.type, r.status, r.progress
FROM sources s LEFT JOIN ranked_jobs r ON r.source_id=s.id AND r.rn=1;
```

### **Оконная функция — рейтинг тегов.**

```
WITH tag_counts AS (
  SELECT t.id,t.display_name,count(DISTINCT dt.document_id) AS documents
  FROM tags t JOIN document_tags dt ON dt.tag_id=t.id GROUP BY t.id
)
SELECT display_name,documents,
       dense_rank() OVER (ORDER BY documents DESC) AS popularity_rank
FROM tag_counts ORDER BY popularity_rank LIMIT 30;
```

Полный набор из 20 запросов указан в приложении.

## 10. Реализация поисковой системы

### 10.1. BM25

Sparse-ветка использует `named vector sparse` и модель `qdrant/bm25` с IDF. BM25 полезен при точных названиях библиотек, идентификаторах, сообщениях об ошибках и совпадающих термах. Недостаток — слабая устойчивость к семантическим перефразировкам.

### 10.2. Vector Search

Dense-ветка получает embeddings через локальный Ollama model `qwen3-embedding:0.6b`. Размерность — 1024, distance — cosine, HNSW parameters — `m=16, ef_construct=100`. Семантический поиск находит близкие по смыслу документы, но может уступать BM25 для редких точных токенов.

### 10.3. Hybrid Search и reranker

BM25 и cosine score имеют разные шкалы и напрямую не суммируются. PyAnswer объединяет ranks формулой weighted Reciprocal Rank Fusion:

$$\text{RRF}(d) = w_{\text{bm25}} / (k + \text{rank}_{\text{bm25}}) + w_{\text{vector}} / (k + \text{rank}_{\text{vector}}).$$

Фактические defaults: `k=60`, оба веса 1; по 60 candidates из sparse/dense и до 80 hybrid candidates. После fusion top candidates передаются Qwen3 reranker. Итог при успешном reranker: `0,85 × reranker_score + 0,15 × normalized_fusion`; при необязательной недоступности допускается явно маркированный fusion-only fallback.

Таблица 2 — Сравнение режимов поиска

| Режим  | Преимущества                    | Недостатки                    | Применение                |
|--------|---------------------------------|-------------------------------|---------------------------|
| BM25   | Точные термы, ошибки, API names | Не понимает дальние перефразы | Точный технический запрос |
| Vector | Семантическая близость          | Возможны неточные lexical     | Описание задачи           |

| Режим  | Преимущества                   | Недостатки        | Применение              |
|--------|--------------------------------|-------------------|-------------------------|
|        |                                | matches           | естественным языком     |
| Hybrid | Совмещает две ветки, reranking | Больше вычислений | Основной режим PyAnswer |

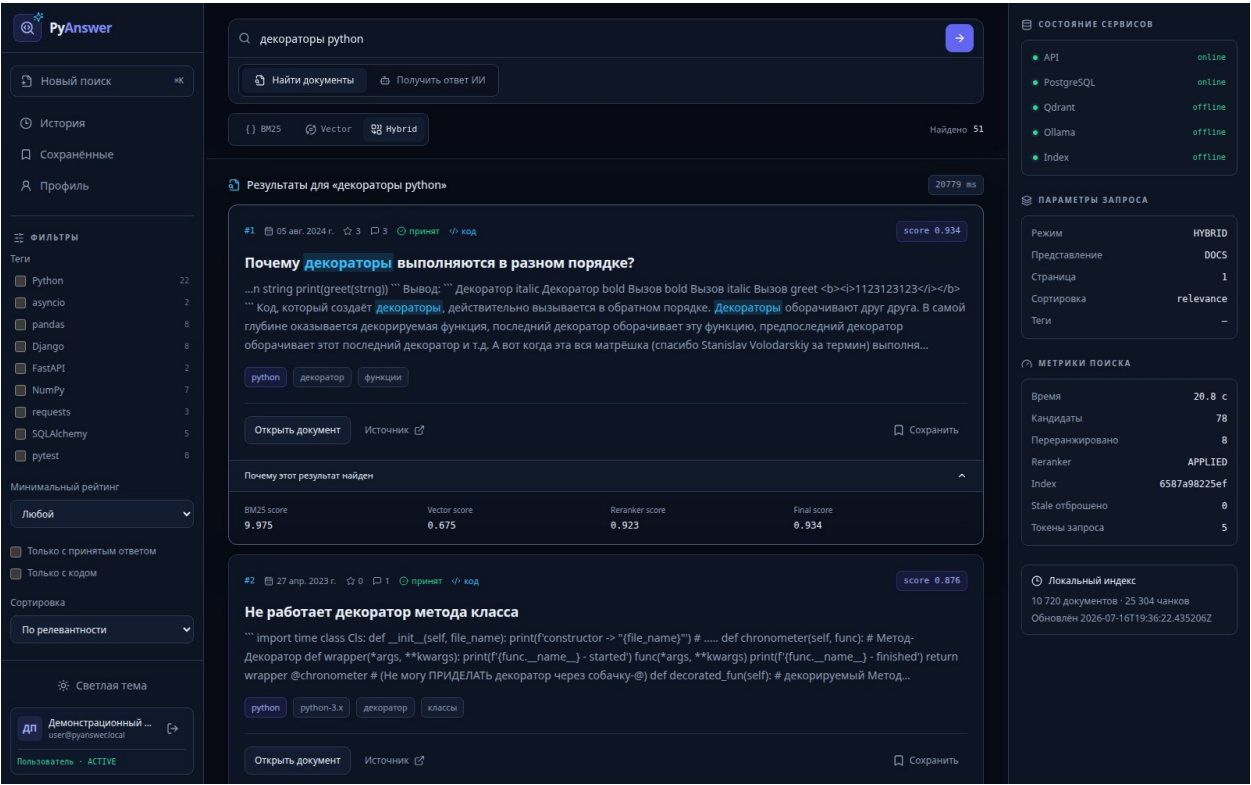


Рисунок 4 — Реальный Hybrid Search и score breakdown

## 11. Qdrant и управление индексом

Qdrant version 1.18.2 хранит один point на индексируемый chunk. Collection содержит named vectors `dense` и `sparse` и payload с IDs, version/status и attribution fields. Физические collections имеют версионированные имена; публичный search использует alias `pyanswer_chunks_current`.

Полная индексация выполняется blue-green: создаётся новая collection, schema и count проверяются, выполняются sample queries, затем alias переключается атомарно. Предыдущая версия может быть сохранена для recovery. Фактический active collection — `pyanswer_chunks_6587a98225ef_20260716182220_05360db8`; 25 214 Qdrant points равны 25214index entries и 25214indexable chunks. `index-consistency-report.json` имеет статус PASS.

## 12. Реализация RAG

RAG pipeline: пользовательский вопрос → hybrid retrieval → reranking → ограничение chunks на документ → context до 5 000 tokens → Ollama qwen3:8b → SSE streaming → citation validation → сохранение rag\_responses и rag\_response\_sources.

Grounded system prompt запрещает использовать instructions из retrieved data и требует ссылки [1]...[N] для существенных утверждений. Source blocks экранируют имитацию citations. После генерации validator проверяет диапазон и наличие citations. Сохраняются model, prompt version/hash, index version, timings, confidence и sources; скрытое reasoning не сохраняется.

Insufficient-context gate срабатывает, если источников меньше 1, context короче 40 tokens, top reranker score ниже 0,25 или обязательный reranker недоступен. В таком случае LLM не вызывается, возвращается безопасный ответ о недостатке контекста. Max sources — 5, max chunks per document — 2.

The screenshot displays the PyAnswer web interface. The main search bar contains the query "Как работают декораторы в Python?". Below the search bar, the retrieval method is set to "Hybrid" (combining BM25 and Vector search). The system has found 3 sources and generated an answer with 98% confidence. The answer text explains Python decorators, their syntax, and how they are applied. The interface includes a sidebar with filters (tags like Python, asyncio, pandas, etc.), a "Состояние сервисов" (Service Status) panel showing API, PostgreSQL, Qdrant, Ollama, and Index status, and a "Параметры запроса" (Query Parameters) panel showing the mode (HYBRID), representation (RAG), page (1), and sorting (relevance). The "Использованные источники" (Used Sources) panel lists three sources: [1] "Почему декораторы выполняются в разном порядке?" (0.978), [2] "Написание декораторов с дополнительной оберткой и без неё" (0.961), and [3] "Как работает декоратор внутри?" (0.891).

Рисунок 5 — Реальный RAG-ответ с проверяемыми источниками

## 13. Пользовательский интерфейс

Frontend реализован на React 18, TypeScript и Vite. Основные страницы: главная, search с переключением document/answer view, документ, login/register, профиль, history, saved, editor и admin. React Query управляет запросами/кэшем; routing использует protected и permission routes.

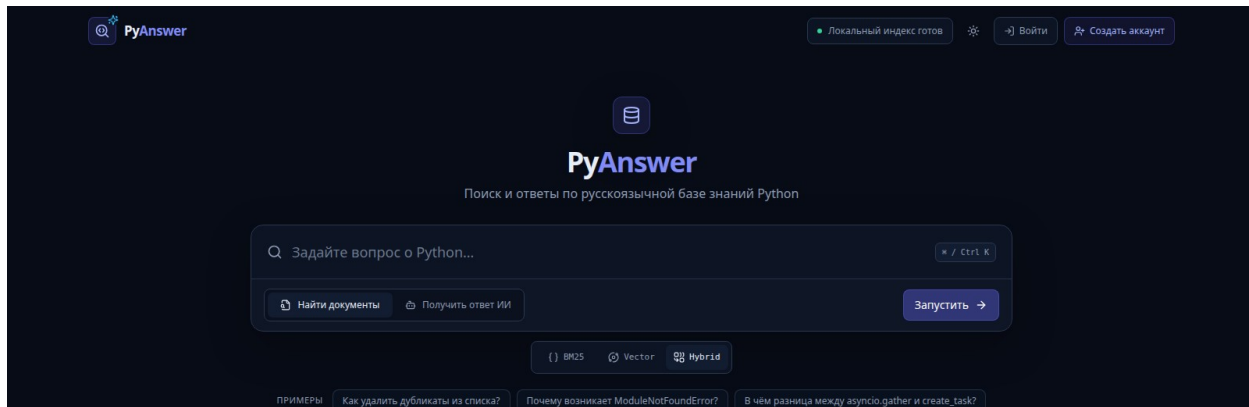


Рисунок 6 — Главная страница PyAnswer

Search page показывает mode, filters, pagination, source attribution и раскрываемый score breakdown. Document page отображает вопрос, ответы, теги, code blocks и внешнюю ссылку. Editor управляет metadata, hide/restore/reindex; admin — users, sources, jobs, audit, indexes, RAG и system status.

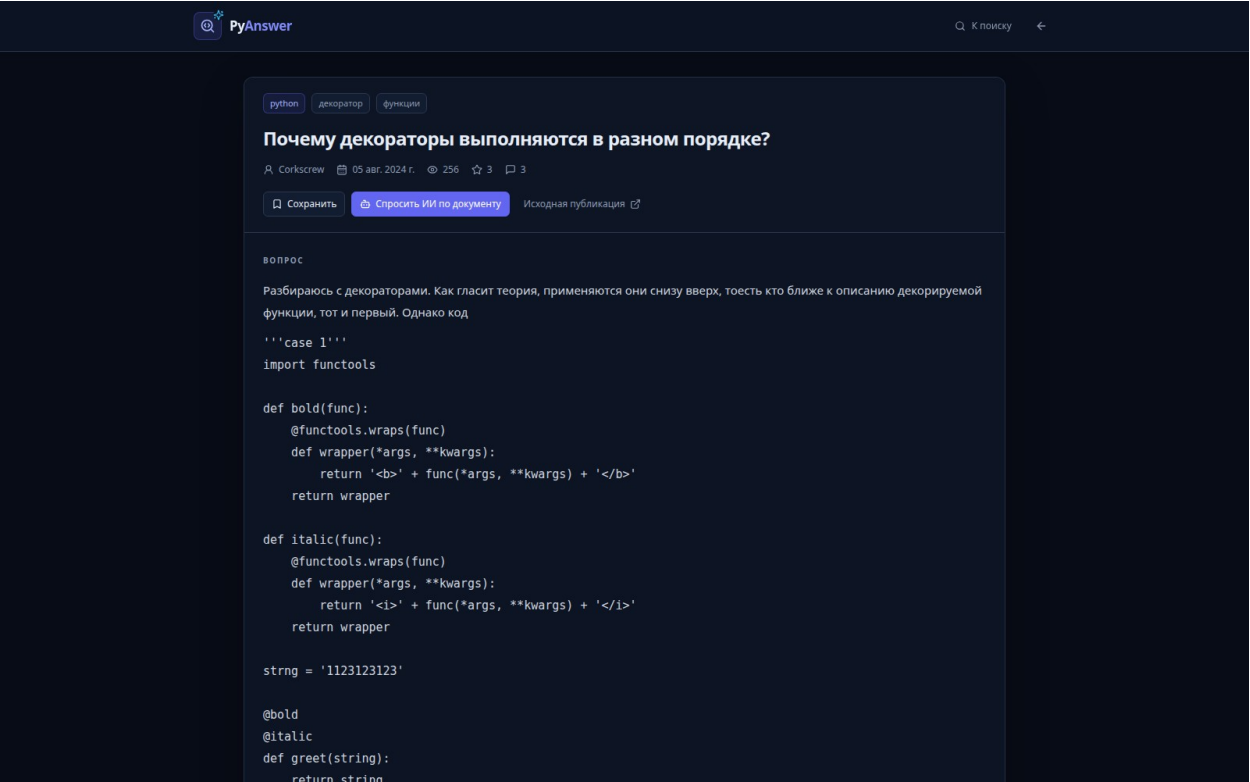


Рисунок 7 — Просмотр документа из корпуса

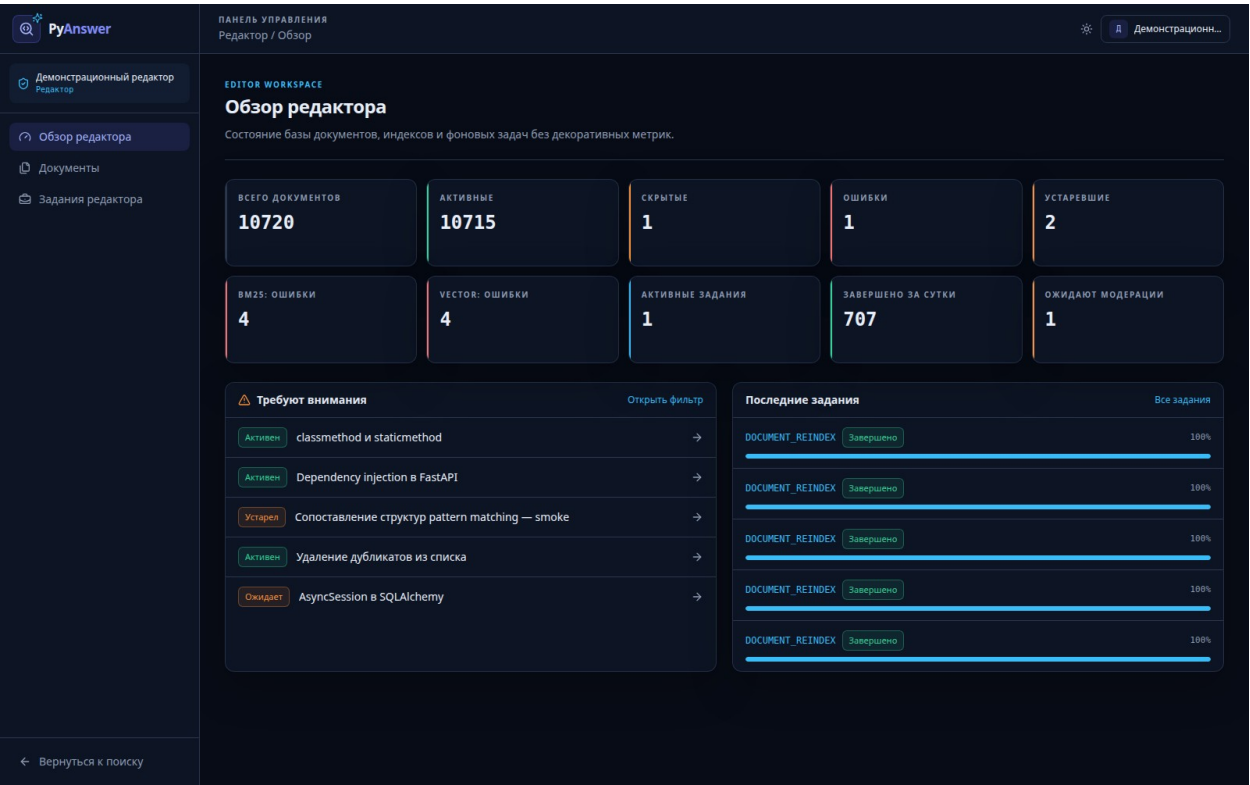


Рисунок 8 — Панель EDITOR

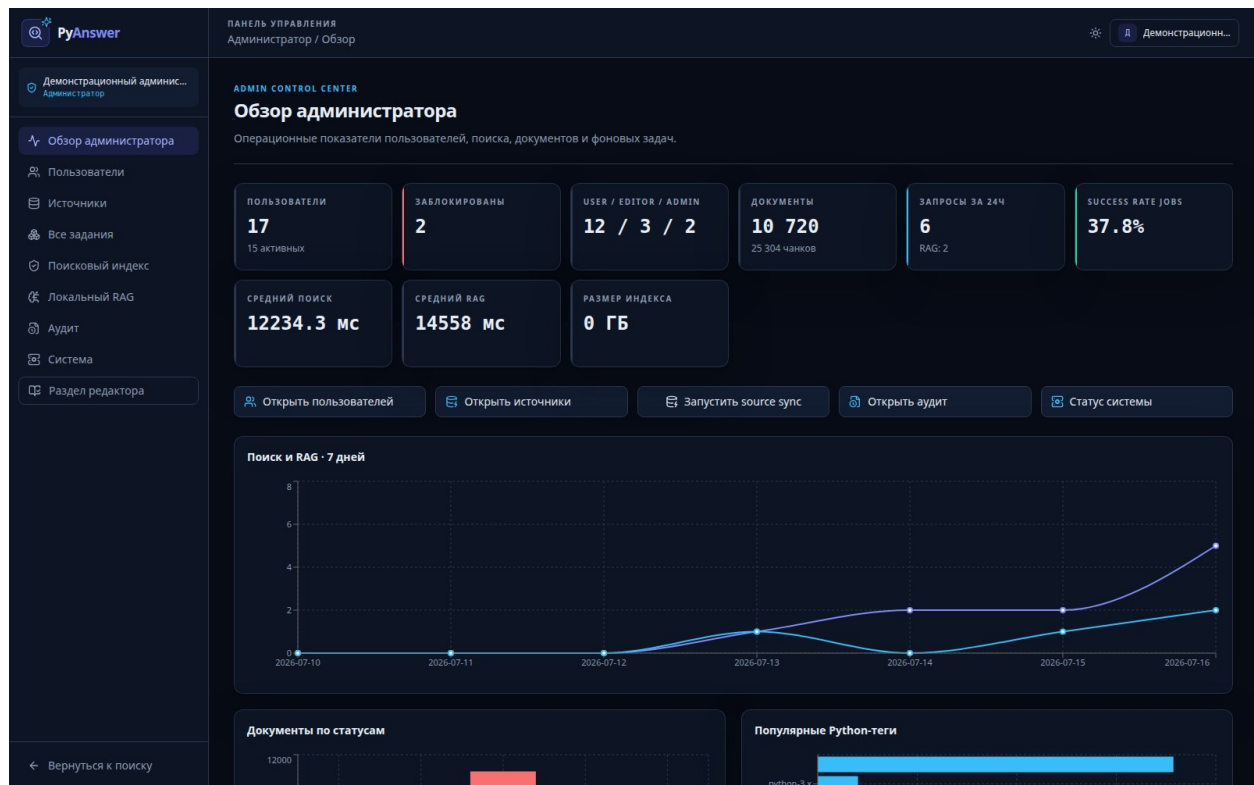


Рисунок 9 — Панель ADMIN

## 14. Модель авторизации

Регистрация всегда создаёт USER и не принимает role. Пароли хэшируются Argon2id; password\_hash хранится только на сервере. Login создаёт server-side session, а browser получает HttpOnly session cookie. CSRF cookie/token и заголовок X-CSRF-Token обязательны для unsafe methods. Logout отзывает текущую session.

Session содержит expiry, remember flag, last seen и account version. После block или смены роли account\_version делает старые sessions недействительными; revoke также проверяется сервером. Blocked user не может войти. Audit sanitizer рекурсивно удаляет credential-like поля из before/after/metadata.

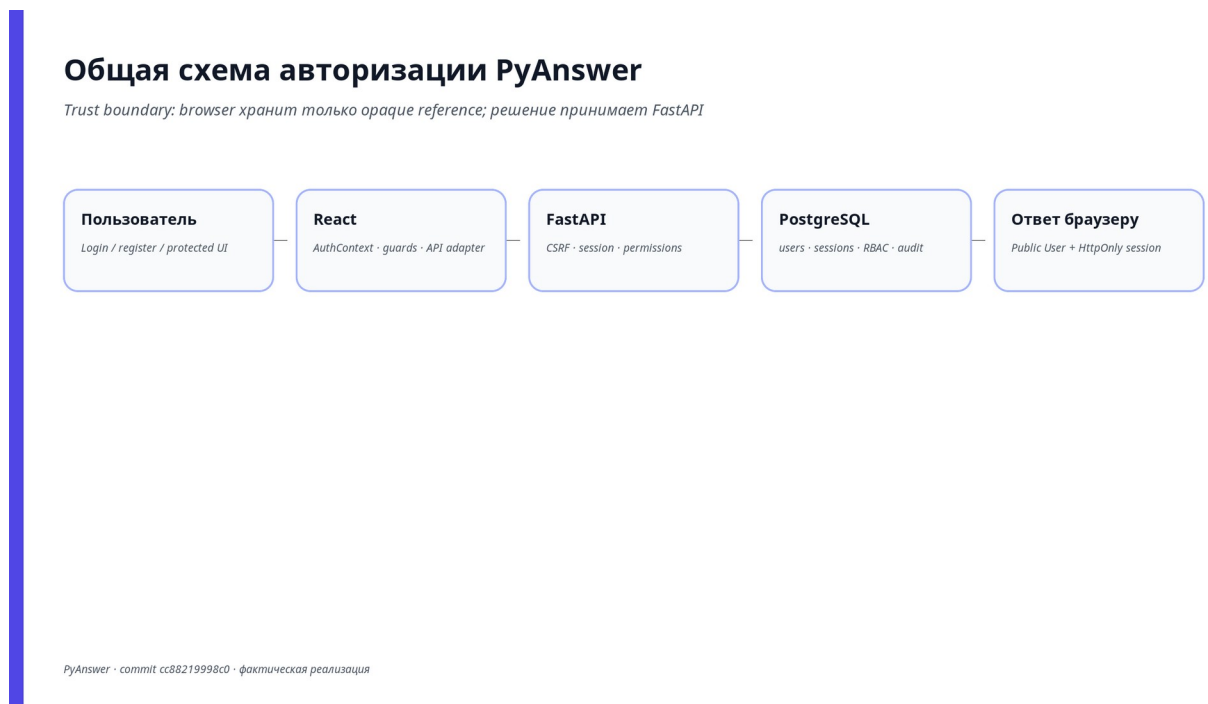


Рисунок 10 — Server-side модель авторизации PyAnswer

## 15. Роли и права доступа

В системе фактически три роли и 20 permissions. USER выполняет поиск/RAG и управляет собственными profile/history/saved/feedback. EDITOR наследует пользовательские возможности и управляет documents/reindex, но не users/sources/system. ADMIN получает системные permissions; self-block/self-role-change запрещены, а последний ACTIVE ADMIN защищён.

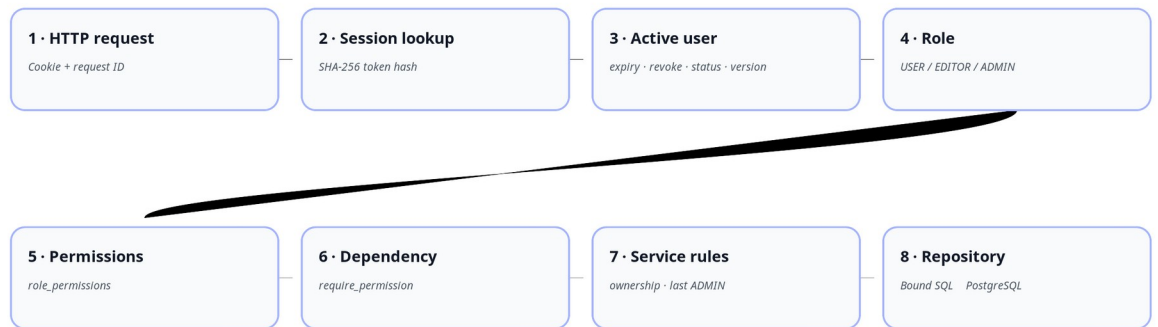
**Таблица 3 — Матрица ролей**

| Возможность                    | USER | EDITOR | ADMIN           |
|--------------------------------|------|--------|-----------------|
| BM25/Vector/Hybrid и RAG       | Да   | Да     | Да              |
| Профиль, history, saved        | Да   | Да     | Да              |
| Editor dashboard               | Нет  | Да     | Да              |
| Metadata, hide/restore/reindex | Нет  | Да     | Да              |
| Users и роли                   | Нет  | Нет    | Ограниченно     |
| Sources, full jobs, indexes    | Нет  | Нет    | Да              |
| Audit                          | Нет  | Нет    | Только просмотр |
| System settings                | Нет  | Нет    | Да              |

Frontend скрывает недоступные функции и показывает 403 route, но обязательная граница находится в backend dependencies require\_permission. Прямой запрещённый API request возвращает 403.

## Проверка permissions на backend

Роль из URL, header или payload не считается доверенной



PyAnswer • commit cc88219998c0 • фактическая реализация

Рисунок 11 — Проверка роли и permission

## 16. API системы

FastAPI публикует API под `/api`; cookie-auth mutation требует CSRF. Основные группы приведены в таблице 4.

**Таблица 4 — Основные endpoints**

| Метод          | Endpoint                                         | Назначение                  | Доступ                   |
|----------------|--------------------------------------------------|-----------------------------|--------------------------|
| GET            | <code>/api/auth/csrf</code>                      | Получить CSRF token         | Public                   |
| POST           | <code>/api/auth/register, /login, /logout</code> | Session lifecycle           | Public/current user      |
| GET/PATCH      | <code>/api/users/me</code>                       | Профиль                     | Authenticated            |
| GET            | <code>/api/search</code>                         | BM25/vector/hybrid          | Policy/<br>SEARCH_USE    |
| POST           | <code>/api/ask, /api/ask/stream</code>           | RAG и SSE                   | Policy/RAG_USE +<br>CSRF |
| GET            | <code>/api/documents/{id}</code>                 | Active document             | Public                   |
| GET/POST/PATCH | <code>/api/editor/*</code>                       | Documents и editor jobs     | EDITOR permissions       |
| GET/PATCH/POST | <code>/api/admin/users/*</code>                  | Users/roles/block           | ADMIN/<br>USERS_MANAGE   |
| GET/POST       | <code>/api/admin/sources/*</code>                | Source test/sync/stop       | ADMIN/<br>SOURCES_MANAGE |
| GET/POST       | <code>/api/admin/jobs/*</code>                   | Retry/cancel/full reindex   | ADMIN_JOBS_MA<br>NAGE    |
| GET            | <code>/api/admin/audit/*</code>                  | Audit read-only             | AUDIT_VIEW               |
| GET/PATCH      | <code>/api/admin/system/*</code>                 | Health/settings             | SYSTEM permissions       |
| GET/POST       | <code>/api/admin/indexes/*</code>                | Blue-green index operations | INDEX permissions        |
| GET            | <code>/api/health/live, /ready</code>            | Healthchecks                | Public/internal          |

Errors возвращаются typed envelope с request ID. Nginx отключает buffering для `/api/ask/stream` и увеличивает streaming timeout.

## 17. Фоновые задания

Таблица `jobs` хранит `type`, `status`, `stage`, `progress`, `payload`, `result`, `checkpoint`, `retry/cancel` и `lease` fields. `job_events` является append-only историей `stage/code/message/metrics`. `worker_instances` хранит `capabilities` и `heartbeat`. Types включают `SOURCE_SYNC`, `DOCUMENT_REPROCESS`, `DOCUMENT_REINDEX`, `FULL_REINDEX`, `HEALTH_CHECK`, `SEARCH_INDEX_VALIDATE` и `SEARCH_INDEX_CLEANUP`.



Рисунок 12 — Жизненный цикл *durable job*

`Claim` выполняется атомарно; `unique partial indexes` предотвращают одновременно активные `duplicate source/full/document reindex jobs`. `Lease` и `heartbeat` позволяют восстановить abandoned job. `Checkpoint` продвигается после `commit`. `Cancellation` является запросом и подтверждается worker-ом в безопасной точке. `Failure isolation` сохраняет ошибку отдельной сущности без остановки всего корпуса.

## 18. Контейнеризация и запуск

`compose.yaml` определяет 8 services и persistent volumes `postgres_data`, `qdrant_data`, `ollama_models`, `reranker_models`. PostgreSQL и Qdrant публикуются только на loopback для диагностики; Ollama/reranker доступны внутри Docker network. Healthchecks используют `pg_isready`, Qdrant TCP, `ollama list`, HTTP readiness и worker heartbeat.

Первый запуск выполняется `make first-run`: создаётся `.env` только при отсутствии, поднимается infrastructure, применяется `alembic upgrade head`, выполняются `idempotent bootstrap/demo seed`, затем запускаются services. Большие модели и полный импорт автоматически не стартуют. Обычная работа: `make demo-up`, статус `make demo-status`, остановка `make demo-down`. Полные команды приведены в README и Makefile.

## 19. Тестирование

Release manifest фиксирует 188 backend tests и 103 frontend tests; backend coverage — 81,125085%. Browser Playwright E2E count в manifest — 0. При этом acceptance выполняет live HTTP search по BM25/vector/hybrid, live RAG с citations и login/logout трёх ролей. Это не следует выдавать за полноценный browser E2E.

**Таблица 5 — Результаты тестирования**

| Вид тестирования               | Результат                          |
|--------------------------------|------------------------------------|
| Backend unit/integration suite | 188 tests, PASS                    |
| Backend coverage               | 81,125085%                         |
| Frontend Vitest                | 103 tests, PASS                    |
| Alembic/head                   | 20260716_0006, PASS                |
| Data consistency               | PASS                               |
| Qdrant consistency             | 25 214 = 25 214, PASS              |
| Acceptance                     | PASS                               |
| Live Search                    | BM25, Vector, Hybrid — PASS        |
| Live RAG                       | HTTP 200, sources/citations — PASS |
| Role login HTTP flow           | USER, EDITOR, ADMIN — PASS         |
| Browser E2E suite              | Не реализован (e2e: 0)             |
| Search/RAG manual evaluation   | Не выполнена: нет reviewed qrels   |

Единичные live latency измерения используются только как smoke evidence и не объявляются benchmark.

## 20. Результаты работы

Таблица 6 — Итоговые показатели PyAnswer

| Показатель                      | Фактическое значение                                | Источник                  |
|---------------------------------|-----------------------------------------------------|---------------------------|
| Реальные обработанные документы | 10 700                                              | corpus manifest           |
| Все документы с demo            | 10 720                                              | corpus manifest           |
| Ответы / выбранные ответы       | 10 261 / 10 165                                     | corpus manifest           |
| Теги                            | 1 475                                               | corpus manifest           |
| Chunks                          | 25 214                                              | corpus manifest           |
| Qdrant points                   | 25 214                                              | index consistency         |
| Revisions                       | 10 700                                              | corpus manifest           |
| PostgreSQL schema               | 29 таблиц, 48 FK                                    | live introspection        |
| Режимы                          | BM25, Vector, Hybrid, RAG                           | acceptance                |
| Роли / permissions              | 3 / 20                                              | bootstrap/RBAC artifacts  |
| Docker services                 | 8                                                   | compose.yaml              |
| Backend/frontend tests          | 188 / 103                                           | release manifest          |
| Coverage                        | 81,125085%                                          | backend coverage artifact |
| Acceptance                      | PASS                                                | acceptance report         |
| Models                          | qwen3-embedding:0.6b; Qwen3 reranker 0.6B; qwen3:8b | release manifest          |

Результат подтверждает работоспособную цепочку от внешнего источника до поиска и grounded RAG в пользовательском интерфейсе.

## 21. Соответствие требованиям задания

Таблица 7 — Доказательства соответствия

| Требование задания | Полученный результат                  | Доказательство                           |
|--------------------|---------------------------------------|------------------------------------------|
| Парсер             | Restart-safe Stack Exchange ingestion | source_sync.py,<br>corpus manifest       |
| Индексы            | Sparse BM25 + dense HNSW              | Qdrant schema, index<br>PASS             |
| Search server      | FastAPI Search API                    | /api/search,<br>integration tests        |
| Top-N/score        | Ranked results и breakdown            | screenshot 04,<br>schemas/search.py      |
| Документ/URL       | Document page + attribution           | screenshot 07,<br>source_url             |
| RAG                | Local Ollama answer with citations    | screenshot 06,<br>acceptance PASS        |
| Hybrid/reranking   | weighted RRF + Qwen3                  | fusion.py,<br>reranker service           |
| UI                 | React/Nginx                           | production<br>Dockerfile,<br>screenshots |
| Auth/RBAC          | Sessions, CSRF, 3 roles               | auth/RBAC<br>tests/artifacts             |
| Background work    | Durable jobs/heartbeats               | jobs schema, worker<br>tests             |
| Deployment         | Docker Compose/Makefile               | config validation,<br>healthchecks       |
| Quality            | Tests, coverage, consistency          | release/acceptance<br>manifests          |

## 22. Полнота фактической реализации

В этом разделе приведена полная инвентаризация функций PyAnswer. Она составлена по текущему коду, PostgreSQL schema, Alembic migrations, API routes, интерфейсу, тестам и итоговым artifacts на commit `60cc0680cba0934d2359701778e743f51f397a8d`. Схожие функции объединены в технические блоки; 120 заданных позиций и добавленный в текущем commit импорт официального Data Dump отдельно отражены в матрице полноты в подразделе 22.12. Показатели, не подтверждённые артефактами, не используются.

### 22.1. Сбор, возобновление и обработка корпуса

Назначение ingestion-контупа — получить воспроизводимый корпус вопросов по Python и не потерять прогресс при временном отказе API или worker. Асинхронный клиент Stack Exchange API 2.3 запрашивает страницы вопросов `ru.stackoverflow` с тегом `python`, учитывает pagination, quota\_remaining, API backoff и HTTP Retry-After. Ограничение частоты вынесено в rate limiter, а сетевые ответы валидируются Pydantic-схемами. Вопросы и ответы запрашиваются пакетно; принятый ответ имеет приоритет, дополнительные ответы выбираются детерминированно по правилам processing.

Независимый bulk-канал использует официальный XML Data Dump. Поточковый `lxml.etree.iterparse` и 7z позволяют не распаковывать весь архив и не загружать нерелевантные строки в память. Импортёр поддерживает ограничение целевого количества, bounded concurrency, parse-only проверку, безопасный JSON report и repair pass для неполных наборов ответов. Он повторно использует тот же ingestion service, поэтому PostgreSQL identity (source\_id, external\_id) и content hash обеспечивают идемпотентность.

Исполнение оформлено как durable PostgreSQL job. Worker атомарно получает задание, записывает lease owner/expiry, посылает heartbeat и

сохраняет checkpoint страницы и позиции только после успешного commit batch. Retry создаёт контролируемую следующую попытку, cancellation проверяется между безопасными этапами. Поэтому процесс можно продолжить после остановки без повторной необратимой обработки уже committed batch.

HTML преобразуется в безопасный текст с сохранением программных фрагментов в fenced code blocks. Нормализация формирует canonical representation; identity определяется парой source/external ID, а SHA-256 canonical content используется для exact deduplication и обнаружения изменений. При изменении создаётся immutable revision. Детерминированный chunker выдаёт стабильные ordinal/key для одной версии документа; единичные ошибки фиксируются отдельно.

**Таблица 8 — Реализация функций сбора и обработки**

| Функции                                             | Технологии и реализация                                                                             | Основные файлы, таблицы и API                                                                                                                              | Подтверждённый результат                                                 |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Парсинг, pagination, quota, backoff, Retry-After    | Async HTTP client Stack Exchange API 2.3, Pydantic, bounded retry и rate limiter                    | integrations/<br>stackexchange/client.py,<br>rate_limiter.py, schemas.<br>py; sources, source_sync_<br>state; POST /api/admin/<br>sources/{id}/test, /sync | 10 700 реальных<br>обработанных<br>документов; corpus<br>manifest        |
| Checkpoint, resume, lease, heartbeat, retry, cancel | Durable queue в PostgreSQL, atomic claim, lease expiry, heartbeat, checkpoint-after-commit          | workers/source_sync.py,<br>workers/runner.py,<br>repositories/jobs.py; jobs,<br>job_events, worker_<br>instances; admin jobs/source<br>endpoints           | Рисунки 2 и 12;<br>worker/job tests; 0<br>открытых ingestion<br>failures |
| Вопросы, ответы и выбор ответа                      | Batch retrieval; accepted answer имеет приоритет, дополнительные ответы выбираются детерминированно | processing/selection.py,<br>services/document_<br>ingestion.py; documents,<br>answers                                                                      | 10 261 ответ и 10<br>165 выбранных<br>ответов                            |
| HTML, код, нормализация, SHA-256, dedup, revisions  | HTML parser, canonical text, cryptographic hash, versioned snapshot                                 | processing/html.py,<br>normalization.py,<br>canonical.py; documents,<br>document_revisions,                                                                | 10 700 revisions;<br>схема ingestion и<br>corpus manifest                |

| Функции                              | Технологии и реализация                                                                            | Основные файлы, таблицы и API                                                                                                 | Подтверждённый результат                                            |
|--------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
|                                      |                                                                                                    | ingestion_failures                                                                                                            |                                                                     |
| Детерминированный чанкинг            | Стабильные границы, ordinal и ключи по версии документа                                            | processing/chunking.py;<br>document_chunks                                                                                    | 25 214 chunks в PostgreSQL                                          |
| Официальный Stack Exchange Data Dump | Потоковый XML через 7z/lxml, отбор python, отдельный проход за ответами и общий ingestion pipeline | backend/app/integrations/<br>stackexchange/dump.py;<br>backend/app/scripts/import_stackexc<br>hange_dump.py; make import-dump | Реализация и unit-тесты существуют; массовая команда не запускалась |

## 22.2. PostgreSQL, ER-модель и миграции

PostgreSQL 16 является источником истины для корпуса, identity/access, jobs, search history, RAG и audit. SQLAlchemy models разделены на content, identity и operations; Alembic последовательно воспроизводит физическую схему до revision 20260716\_0006. Фактическая схема включает 29 таблиц с alembic\_version и 48 FK. Полная ERD показывает PK, FK, UNIQUE, nullable, N:M bridge tables и self-references.

Связь 1:1 представлена users — user\_preferences; 1:N — sources — documents, documents — answers/chunks/revisions, jobs — job\_events; N:M — roles/permissions, documents/tags и users/saved documents через таблицы связей. Пользователи, роли и server-side sessions хранятся отдельно; jobs и audit сохраняют операционную историю, а rag\_responses/rag\_response\_sources — результат генерации и использованные доказательства.

**Таблица 9 — Реализация хранения данных**

| Назначение                  | Реализация                                                                 | Код/таблицы                                                                                           | Доказательство                            |
|-----------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------|
| Корпус и история            | Нормализованная PostgreSQL schema с FK и revisions                         | db/models/content.py;<br>sources, documents, answers,<br>tags, document_chunks,<br>document_revisions | Рисунок 3 и приложение А; corpus manifest |
| Identity и пользовательские | Users, roles, permissions, sessions, preferences, history, saved, feedback | db/models/identity.py;<br>соответствующие API /api/                                                   | Auth/RBAC tests и реальные экраны         |

| Назначение                     | Реализация                                                               | Код/таблицы                                        | Доказательство                     |
|--------------------------------|--------------------------------------------------------------------------|----------------------------------------------------|------------------------------------|
| функции                        |                                                                          | users, /api/history, /api/saved, /api/feedback     |                                    |
| Operations, search, RAG, audit | Durable jobs/events, versions/entries/runs, RAG sources, sanitized audit | db/models/operations.py; admin/search/RAG services | 29 таблиц, 48 FK; ERD и acceptance |
| Эволюция schema                | Шесть Alembic revisions от initial schema до local RAG                   | backend/alembic/versions/                          | Alembic head 20260716_0006         |

### 22.3. Индексирование, три режима поиска и ранжирование

Поиск работает по chunks активных документов. Sparse-ветвь формирует BM25-подобный Qdrant sparse vector из локальной токенизации; dense-ветвь вызывает локальную embedding-модель qwen3-embedding:0.6b через Ollama и использует cosine HNSW. В Qdrant point одновременно хранятся named sparse и dense vectors и payload, связывающий point с PostgreSQL chunk/document/version.

Search API поддерживает режимы BM25, Vector и Hybrid, top-N, pagination и фильтры по источнику, тегам, наличию принятого ответа, кода и статусу документа. Hybrid объединяет ранги weighted Reciprocal Rank Fusion с  $k=60$ ; исходные score разных шкал напрямую не складываются. Затем локальный Qwen/Qwen3-Reranker-0.6B пересчитывает релевантность кандидатов. При включённом reranker итоговый score формируется из 0,85 нормализованного reranker score и 0,15 нормализованного fusion score; breakdown сохраняет вклад ветвей.

Full reindex использует blue-green схему: создаётся новая collection, индексируются chunks, выполняется validation, после чего alias pyanswer\_chunks\_current атомарно переключается. Старый индекс остаётся доступен до успешного switch. Consistency check сопоставляет active entries/chunks и Qdrant points.

**Таблица 10 — Реализация поиска и индекса**

| Функции                                | Технологии и параметры                                                             | Файлы/таблицы/API                                                                                                            | Результат и доказательство                                                  |
|----------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| BM25, Vector, Hybrid                   | Qdrant sparse; Ollama embedding; cosine HNSW; weighted RRF k=60                    | integrations/qdrant/<br>sparse.py, search/query.py, Рисунки 4–5; live<br>search/fusion.py; GET /<br>api/search               | Search PASS                                                                 |
| top-N, score,<br>breakdown, filters    | FastAPI query schema, SQL/Qdrant<br>filters, typed response, React result<br>cards | services/search.py,<br>search/filters.py,<br>ResultCard.tsx,<br>FiltersPanel.tsx; search_<br>runs                            | Screenshot 04_<br>Hybrid_score_<br>breakdown.png;<br>frontend/backend tests |
| Локальный reranker и<br>финальный rank | Отдельный HTTP service с Qwen3<br>Reranker; bounded candidate list                 | integrations/reranker/<br>client.py, backend/app/<br>reranker.py; Search API                                                 | Acceptance PASS;<br>release manifest<br>фиксирует модель                    |
| Blue-green, alias и<br>validation      | Versioned collections, atomic alias<br>switch и cleanup                            | services/search_index.py,<br>workers/indexer.py;<br>search_index_versions,<br>search_index_entries; /<br>api/admin/indexes/* | 25 214 chunks = 25<br>214 entries = 25 214<br>points, consistency<br>PASS   |

## 22.4. Локальный RAG и защита контекста

RAG выполняется после retrieval/reranking. Сервис выбирает ограниченное число разнообразных chunks, формирует контекст с явными source identifiers и передаёт его локальной generation-модели qwen3:8b через Ollama. Ответ можно получить обычным POST или потоком Server-Sent Events. Browser отображает прогресс, текст, источники и ссылки; завершённый ответ сохраняется вместе с ordered source rows.

Grounding gate проверяет минимальный объём и reranker confidence: при недостатке данных возвращается insufficient\_context, и generation model не вызывается. Prompt отделяет system instruction от недоверенного retrieved context и прямо запрещает выполнять инструкции внутри документов. Citation parser принимает только разрешённые identifiers; неизвестные/неподтверждённые ссылки не считаются источниками. Retrieval и context builder исключают hidden, неактуальные и не соответствующие active

index version chunks.

Таблица 11 — Реализация RAG

| Функции                                 | Реализация                                                                       | Код/данные/API                                             | Доказательство                                     |
|-----------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------|
| Local LLM Qwen3                         | Ollama, модель qwen3:8b, без внешнего cloud LLM API                              | integrations/llm/ollama.py, services/rag.py; POST /api/ask | Release manifest; live RAG HTTP 200                |
| Context и grounded generation           | Candidate/rerank limits, max context, per-document diversity, structured sources | rag/context.py, rag/confidence.py, rag/prompt.py           | Screenshot Рисунок 6; RAG tests                    |
| Streaming SSE                           | События статуса, token/delta, sources, done/error                                | POST /api/ask/stream, useRagStream.ts, Nginx no-buffering  | Screenshot 05_RAG_streaming.png; acceptance PASS   |
| Citations и persistence                 | Allow-list validation и связь ответа с document/chunk                            | rag/citations.py; rag_responses, rag_response_sources      | Valid citations в acceptance report; ERD           |
| Insufficient context и context security | Gate до LLM; active/visible filters; untrusted-context delimiters                | integrations/llm/gate.py, rag/prompt.py, search filters    | Recorded insufficient-context scenario; unit tests |

22.5. React-интерфейс и пользовательские функции

Frontend реализован как React + TypeScript SPA, собираемая Vite и отдаваемая Nginx. AppRoutes.tsx определяет публичные и защищённые маршруты; httpApi.ts централизует cookie credentials, CSRF и typed errors. Главная страница объясняет режимы системы; Search page предоставляет query, режим, фильтры, pagination, top-N и score breakdown; Document page отображает обсуждение и attribution.

После входа пользователь работает с Profile, History и Saved pages. Preferences обновляются через профиль, история создаётся сервером для search/RAG, saved documents реализуют N:M user-document, feedback привязан к пользователю и RAG response. Состояния loading/empty/error и 401/403 представлены явно.

Таблица 12 — Реализация frontend и пользовательских

## ВОЗМОЖНОСТЕЙ

| Возможности                       | Технологии                                       | Файлы/API/таблицы                                                             | Доказательство                     |
|-----------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------|------------------------------------|
| React, TypeScript, Vite, главная  | SPA routing, typed components и production build | frontend/src/main.tsx, routes/AppRoutes.tsx, pages/HomePage.tsx, package.json | Рисунок 4; 103 frontend tests PASS |
| Search, filters, score breakdown  | Controlled URL params, result cards, pagination  | SearchPage.tsx, FiltersPanel.tsx, ResultCard.tsx; GET /api/search             | Screenshots 02–04                  |
| Документ                          | Question/answers/tags/code/source link           | DocumentPage.tsx, DocumentThread.tsx; GET /api/documents/{id}                 | Рисунок 7                          |
| Profile/preferences/history/saved | Authenticated pages и owner-scoped APIs          | ProfilePage.tsx, HistoryPage.tsx, SavedPage.tsx; user/history/saved tables    | Screenshot 08; component/API tests |
| Feedback                          | Upsert/get/delete собственной оценки             | api/routes/feedback.py; feedback; /api/feedback                               | Backend route/service tests        |

## 22.6. Аутентификация, sessions и RBAC

Регистрация создаёт только USER. Пароли хэшируются Argon2id. Session token является opaque random secret: клиент получает его только в HttpOnly-cookie, а PostgreSQL хранит hash token, expiry, revoke state и account version. Unsafe cookie-auth requests требуют double-submit CSRF token в cookie/header. Logout отзывает текущую session; expired/revoked sessions отклоняются; block или role change увеличивает account version и инвалидирует старые sessions.

RBAC состоит из ролей USER, EDITOR и ADMIN и 20 permissions. require\_permission проверяет право в backend для каждого management endpoint. Frontend ProtectedRoute/PermissionRoute улучшает UX, но не является security boundary. Отсутствие authentication даёт 401, недостаток permission — 403. Bootstrap использует принцип минимальных привилегий;

self-demotion/self-block запрещены, последний ACTIVE ADMIN защищён транзакционной проверкой.

**Таблица 13 — Реализация security и доступа**

| Функции                          | Реализация                                                                  | Файлы/таблицы/API                                                                    | Доказательство                            |
|----------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------|
| Register/login/logout и Argon2id | FastAPI auth service, secure hash/verify                                    | core/security.py, services/auth.py; users; /api/auth/*                               | Рисунок 10; auth tests и login acceptance |
| Opaque server-side sessions      | Random token, DB token hash, HttpOnly-cookie, expiry/revoke/account version | repositories/sessions.py; sessions; /api/auth/me                                     | session-flow diagram; tests               |
| CSRF                             | Token cookie + X-CSRF-Token на unsafe requests                              | core/csrf.py, frontend/src/api/httpApi.ts; /api/auth/csrf                            | CSRF integration tests                    |
| USER/EDITOR/ADMIN и permissions  | Backend dependencies, frontend route guards, 401/403                        | core/permissions.py, api/dependencies.py, features/auth/*; roles/permissions bridges | Рисунок 11; матрица ролей; 403 screenshot |
| Block/invalidation/last admin    | Account version bump и server-side safety checks                            | services/admin_users.py; admin user endpoints                                        | Admin security diagram и backend tests    |

## 22.7. EDITOR, ADMIN и безопасный audit

EDITOR получает dashboard и owner-independent управление корпусом в пределах document permissions: список и карточку документа, chunks/revisions/failures, обновление управленческих metadata и tags, hide/restore, bulk actions и enqueue document reindex. Hide исключает документ из публичной выдачи и следующего активного индекса; restore возвращает его к обработке.

ADMIN управляет users/roles/block state, sources, source test/sync/stop, jobs retry/cancel, ingestion diagnostics, index full-reindex/validate/cleanup, RAG diagnostics и system status/settings. Каждая mutation проходит permission и CSRF checks и записывает audit event с actor/action/target/reason/request ID. Sanitizer рекурсивно удаляет password, token, cookie, authorization и другие

credential-like поля из snapshots и metadata.

**Таблица 14 — Реализация панелей управления**

| Возможности                                            | Реализация                                                   | Файлы/API/таблицы                                                                 | Доказательство                        |
|--------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------|
| EDITOR documents, hide/restore, metadata/tags, reindex | FastAPI editor service + React management pages              | services/editor.py, pages/editor/; documents, tags, revisions, jobs; /api/editor/ | Рисунок 8; demo video editor scenario |
| ADMIN users/roles/block                                | Permission-guarded user administration                       | services/admin_users.py, AdminUsersPage.tsx; /api/admin/users/*                   | Рисунок 9; auth screenshots 07–08     |
| Sources и sync                                         | Connection test, checkpoint, enqueue/stop sync               | services/admin_sources.py, SourcesPage.tsx; /api/admin/sources/*                  | Screenshot 11; jobs/source tests      |
| Jobs и indexes                                         | Events, retry/cancel, blue-green, validate/cleanup           | admin_jobs.py, admin_indexes.py, JobsPage.tsx, IndexesPage.tsx                    | Screenshot 12; consistency PASS       |
| Audit/system/RAG diagnostics                           | Read-only audit, health/settings, diagnostic RAG             | admin_audit.py, admin_system.py, admin_rag.py; audit_events, system_settings      | Screenshots 13–14; acceptance         |
| Audit sanitizer                                        | Recursive removal of credential-like keys before persistence | services/audit.py                                                                 | Security report и sanitizer tests     |

## 22.8. Контейнеризация, устойчивость и единый origin

Docker Compose связывает восемь фактических services: PostgreSQL 16, Qdrant 1.18.2, Ollama, FastAPI backend, ingestion worker, indexing worker, reranker и frontend/Nginx. Named volumes сохраняют PostgreSQL data, Qdrant index, Ollama models и reranker model cache. Healthchecks проверяют database readiness, Qdrant/Ollama, API live/ready, worker heartbeat и frontend. Nginx отдаёт SPA и reverse-proxy /api, поэтому browser использует один origin и не получает прямой доступ к внутренним сервисам.

**Таблица 15 — Реализация эксплуатации**

| Функции              | Файлы и технологии                                                    | Результат/<br>доказательство                   |
|----------------------|-----------------------------------------------------------------------|------------------------------------------------|
| Compose и 8 services | compose.yaml, service Dockerfiles                                     | Compose config valid; release manifest         |
| Persistent volumes   | postgres_data, qdrant_data, ollama_models, reranker_models            | Данные и модели переживают restart контейнеров |
| Healthchecks         | Docker healthchecks, /api/health/live, /ready, worker health          | Screenshot 14; acceptance PASS                 |
| Nginx/same origin    | frontend/nginx.conf, production Dockerfile, /api проху и SSE settings | Browser обращается только к frontend origin    |

## 22.9. Проверки, реальная работа и измеренные показатели

Автоматические проверки разделены на backend pytest и frontend Vitest. Release manifest подтверждает 188 backend и 103 frontend tests, backend coverage 81,125085%. Qdrant integration tests проверяют schema/upsert/search; index consistency сопоставляет PostgreSQL chunks/entries с points. Acceptance выполняет реальные HTTP BM25, Vector, Hybrid и RAG-запросы, citations и вход ролей. Browser E2E test suite отсутствует (e2e: 0), поэтому реальные screenshots и demo video являются отдельным ручным подтверждением интерфейса, а не подменяются термином E2E.

**Таблица 16 — Проверяемость результата**

| Проверка                       | Технология/артефакт                     | Фактический<br>результат                         |
|--------------------------------|-----------------------------------------|--------------------------------------------------|
| Backend tests/coverage         | pytest, coverage JSON                   | 188 PASS;<br>81,125085%                          |
| Frontend tests                 | Vitest/Testing Library                  | 103 PASS                                         |
| Qdrant integration/consistency | Qdrant test marker и consistency script | 25 214 = 25 214 = 25<br>214; PASS                |
| Acceptance/live Search/RAG     | finalization/acceptance.py, HTTP probes | Final acceptance<br>PASS; RAG citations<br>valid |

| Проверка    | Технология/артефакт        | Фактический результат                    |
|-------------|----------------------------|------------------------------------------|
| Корпус      | corpus manifest            | 10 700 real; 10 720 total; 25 214 chunks |
| UI evidence | 14 screenshots и WebM demo | Реальный HTTP, mocks disabled            |

## 22.10. SQL, документация и материалы защиты

Инструкция запуска находится в README/Makefile; отдельный read-only SQL-набор демонстрирует JOIN, GROUP BY, подзапрос, CTE и оконные функции на реальных таблицах. Документация сервисов и скриптов представлена итоговым PDF в каталоге отчет. Демонстрационное видео отчет/PyAnswer\_demo\_full.webm длительностью 05:15 показывает Search, RAG, USER, EDITOR и ADMIN по реальному HTTP. Презентация существует в PDF и PPTX.

На момент подготовки данного отчёта безопасный ZIP-архив исходного кода в рабочем дереве отсутствует: сохранён отчёт о коде сервисов/скриптов и доступен GitHub commit. Поэтому пункт об архиве отмечен «Выполнено частично», без выдуманного пути или checksum. Это единственное расхождение с перечнем дополнительных артефактов.

**Таблица 17 — Документация и артефакты**

| Материал                | Фактическое расположение                        | Подтверждение                          |
|-------------------------|-------------------------------------------------|----------------------------------------|
| Инструкция запуска      | README.md, Makefile                             | Копируемые команды и health flow       |
| SQL-примеры             | docs/final/sql-demo.sql, docs/final/sql-demo.md | JOIN, GROUP BY, subquery, CTE, windows |
| Код сервисов и скриптов | отчет/Код_сервисов_и_скриптов_PyAnswer.pdf      | Итоговый документ существует           |
| Видео                   | отчет/PyAnswer_demo_full.webm                   | 05:15, 1440×900, real HTTP             |

| Материал     | Фактическое расположение              | Подтверждение               |
|--------------|---------------------------------------|-----------------------------|
| Презентация  | отчет/PyAnswer_Презентация.pdf, .pptx | Итоговые файлы существуют   |
| Исходный ZIP | В текущем рабочем дереве отсутствует  | Статус «Выполнено частично» |

## 22.11. Индивидуальный вклад автора

Проект выполнен индивидуально Горбачёвым Вадимом Александровичем. Индивидуальный вклад охватывает анализ кейса; проектирование архитектуры и PostgreSQL schema; разработку React frontend и FastAPI backend; ingestion через API и официальный Data Dump, нормализацию, deduplication и chunking; durable workers; Qdrant hybrid search и reranker; локальный RAG; Docker integration; автоматические проверки; подготовку ERD, screenshots, video, presentation и документации. Репозиторий фиксирует результат commit 60cc0680cba0934d2359701778e743f51f397a8d; отдельное описание вклада находится в docs/final/contribution.md.

## 22.12. Таблица полноты реализации

**Таблица 18 — Полнота реализации 121 пункта**

| № | Реализованный пункт                                        | Где описан в отчёте | Доказательство                                        | Статус    |
|---|------------------------------------------------------------|---------------------|-------------------------------------------------------|-----------|
| 1 | Парсинг Stack Overflow на русском через Stack Exchange API | 6; 22.1             | integrations/stackexchange/client.py; corpus manifest | Выполнено |
| 2 | Работа с пагинацией API                                    | 6; 22.1             | Stack Exchange client pagination tests                | Выполнено |
| 3 | Контроль quota, backoff и Retry-After                      | 6; 22.1             | rate_limiter.py; client tests                         | Выполнено |
| 4 | Сохранение checkpoint и продолжение импорта                | 6; 17; 22.1         | source_sync_state; workers/source_sync.py             | Выполнено |
| 5 | Durable background jobs                                    | 17; 22.1            | jobs, job_events; Рисунок 12                          | Выполнено |
| 6 | Lease и heartbeat фоновых заданий                          | 17; 22.1            | worker/job tests; worker_instances                    | Выполнено |

| №  | Реализованный пункт                      | Где описан в отчёте | Доказательство                              | Статус    |
|----|------------------------------------------|---------------------|---------------------------------------------|-----------|
| 7  | Retry и cancellation заданий             | 17; 22.1            | POST /api/admin/jobs/{{id}}/retry и /cancel | Выполнено |
| 8  | Получение вопросов и ответов             | 6; 22.1             | 10 261 answers; corpus manifest             | Выполнено |
| 9  | Выбор принятого и дополнительных ответов | 6; 22.1             | processing/selection.py; 10 165 selected    | Выполнено |
| 10 | Очистка HTML                             | 7; 22.1             | processing/html.py; processing tests        | Выполнено |
| 11 | Сохранение блоков программного кода      | 7; 22.1             | processing/html.py; Рисунок 7               | Выполнено |
| 12 | Нормализация данных                      | 7; 22.1             | processing/normalization.py                 | Выполнено |
| 13 | Дедупликация документов                  | 7; 22.1             | processing/canonical.py; ingestion service  | Выполнено |
| 14 | SHA-256 и контроль изменений содержимого | 7; 22.1             | documents.content_hash; revisions           | Выполнено |
| 15 | Хранение истории изменений документов    | 8; 22.2             | document_revisions; ERD                     | Выполнено |
| 16 | Детерминированный чанкинг                | 7; 22.1             | processing/chunking.py; 25 214 chunks       | Выполнено |
| 17 | Хранение корпуса в PostgreSQL            | 8; 22.2             | corpus tables; corpus manifest              | Выполнено |
| 18 | Хранение пользователей, ролей и сессий   | 8; 14–15; 22.2      | identity models; auth tests                 | Выполнено |
| 19 | Хранение jobs, audit и RAG-результатов   | 8; 17; 22.2         | operations models; ERD                      | Выполнено |
| 20 | Полная ER-модель базы данных             | 8; приложение А     | отчет/ER/PyAnswer_ERD_Full.pdf              | Выполнено |
| 21 | Связи 1:1, 1:N и N:M                     | 8; 22.2             | 29 tables, 48 FK; ERD                       | Выполнено |
| 22 | Alembic migrations                       | 4; 8; 22.2          | revision 20260716_0006                      | Выполнено |
| 23 | Полнотекстовый поиск BM25                | 10; 22.3            | qdrant/sparse.py; screenshot 02             | Выполнено |
| 24 | Векторный поиск                          | 10; 22.3            | GET /api/search?mode=vector; screenshot 03  | Выполнено |
| 25 | Использование embedding-модели           | 10; 22.3            | qwen3-embedding:0.6b; release               | Выполнено |

| №  | Реализованный пункт                 | Где описан в отчёте | Доказательство                        | Статус    |
|----|-------------------------------------|---------------------|---------------------------------------|-----------|
|    |                                     |                     | manifest                              |           |
| 26 | Dense cosine HNSW                   | 10; 11; 22.3        | Qdrant schema; index tests            | Выполнено |
| 27 | Sparse и dense vectors в Qdrant     | 11; 22.3            | named vectors; qdrant/schema.py       | Выполнено |
| 28 | Три режима BM25, Vector, Hybrid     | 10; 22.3            | Search API; live acceptance PASS      | Выполнено |
| 29 | Выдача top-N результатов            | 10; 22.3            | GET /api/search; SearchPage           | Выполнено |
| 30 | Отображение score                   | 10; 13; 22.3        | ResultCard.tsx; screenshots           | Выполнено |
| 31 | Score breakdown                     | 10; 13; 22.3        | Рисунок 5; search schemas             | Выполнено |
| 32 | Фильтрация результатов              | 10; 13; 22.3        | search/filters.py; FiltersPanel.tsx   | Выполнено |
| 33 | Weighted Reciprocal Rank Fusion     | 10; 22.3            | search/fusion.py; k=60                | Выполнено |
| 34 | Локальный reranker                  | 10; 22.3            | Qwen3-Reranker-0.6B; release manifest | Выполнено |
| 35 | Итоговое ранжирование результатов   | 10; 22.3            | fusion/reranker score; acceptance     | Выполнено |
| 36 | Blue-green построение индекса       | 11; 22.3            | services/search_index.py              | Выполнено |
| 37 | Alias активной коллекции Qdrant     | 11; 22.3            | pyanswer_chunks_current; consistency  | Выполнено |
| 38 | Сверка chunks и Qdrant points       | 11; 19; 22.3        | 25 214 = 25 214; PASS                 | Выполнено |
| 39 | Локальная LLM через Ollama          | 12; 22.4            | integrations/llm/ollama.py            | Выполнено |
| 40 | Использование модели Qwen3          | 12; 22.4            | qwen3:8b; release manifest            | Выполнено |
| 41 | Retrieval-Augmented Generation      | 12; 22.4            | services/rag.py; live RAG PASS        | Выполнено |
| 42 | Формирование контекста              | 12; 22.4            | rag/context.py; RAG tests             | Выполнено |
| 43 | Streaming ответа через SSE          | 12; 22.4            | POST /api/ask/stream; screenshot 05   | Выполнено |
| 44 | Отображение источников              | 12; 13; 22.4        | Рисунок 6; RagAnswer.tsx              | Выполнено |
| 45 | Проверка citations                  | 12; 22.4            | rag/citations.py; acceptance PASS     | Выполнено |
| 46 | Сохранение RAG-ответов и источников | 8; 12; 22.4         | rag_responses, rag_response_sources   | Выполнено |
| 47 | Режим insufficient context          | 12; 22.4            | confidence gate; demo video           | Выполнено |

| №  | Реализованный пункт                       | Где описан в отчёте | Доказательство                        | Статус    |
|----|-------------------------------------------|---------------------|---------------------------------------|-----------|
| 48 | Защита от скрытых/неактуальных документов | 12; 22.4            | active/visible filters; search tests  | Выполнено |
| 49 | Защита от prompt injection в context      | 12; 22.4            | rag/prompt.py; RAG tests              | Выполнено |
| 50 | React frontend                            | 13; 22.5            | frontend/src; production build        | Выполнено |
| 51 | TypeScript и Vite                         | 13; 22.5            | frontend/package.json, Vite config    | Выполнено |
| 52 | Главная страница                          | 13; 22.5            | HomePage.tsx; screenshot 01           | Выполнено |
| 53 | Поисковый интерфейс                       | 13; 22.5            | SearchPage.tsx; screenshots 02–04     | Выполнено |
| 54 | Страница документа                        | 13; 22.5            | DocumentPage.tsx; Рисунок 7           | Выполнено |
| 55 | Профиль пользователя                      | 13; 22.5            | ProfilePage.tsx; screenshot 08        | Выполнено |
| 56 | История запросов                          | 13; 22.5            | HistoryPage.tsx; /api/history         | Выполнено |
| 57 | Сохранённые документы                     | 13; 22.5            | SavedPage.tsx; /api/saved             | Выполнено |
| 58 | Пользовательские настройки                | 13; 22.5            | user_preferences; PATCH /api/users/me | Выполнено |
| 59 | Feedback на результаты и ответы           | 13; 22.5            | /api/feedback; feedback               | Выполнено |
| 60 | Регистрация пользователя                  | 14; 22.6            | POST /api/auth/register; auth tests   | Выполнено |
| 61 | Вход и выход                              | 14; 22.6            | login/logout endpoints; acceptance    | Выполнено |
| 62 | Хеширование паролей Argon2id              | 14; 22.6            | core/security.py                      | Выполнено |
| 63 | Server-side opaque sessions               | 14; 22.6            | session repository; session diagram   | Выполнено |
| 64 | Хранение только hash session token        | 14; 22.6            | sessions.token_hash; ERD              | Выполнено |
| 65 | HttpOnly-cookie                           | 14; 22.6            | auth service/config; security tests   | Выполнено |
| 66 | CSRF-защита                               | 14; 22.6            | core/csrf.py; CSRF tests              | Выполнено |
| 67 | Отзыв и истечение сессий                  | 14; 22.6            | sessions repository; logout tests     | Выполнено |
| 68 | Блокировка пользователей                  | 14–15; 22.6         | POST /api/admin/users/{id}/block      | Выполнено |
| 69 | Инвалидирование старых сессий             | 14; 22.6            | account_version; tests                | Выполнено |
| 70 | Роли USER, EDITOR и ADMIN                 | 15; 22.6            | bootstrap; матрица полей              | Выполнено |

| №  | Реализованный пункт                    | Где описан в отчёте | Доказательство                          | Статус    |
|----|----------------------------------------|---------------------|-----------------------------------------|-----------|
| 71 | Система permissions                    | 15; 22.6            | 20 permissions; core/permissions.py     | Выполнено |
| 72 | Проверка permissions на backend        | 15; 22.6            | require_permission; RBAC tests          | Выполнено |
| 73 | Защищённые маршруты frontend           | 15; 22.6            | ProtectedRoute/PermissionRoute tests    | Выполнено |
| 74 | Обработка ошибок 401 и 403             | 15; 22.6            | ApiError.ts; 403 screenshot             | Выполнено |
| 75 | Принцип минимальных привилегий         | 15; 22.6            | role-permission matrix                  | Выполнено |
| 76 | Защита последнего активного ADMIN      | 15; 22.6            | admin_users service tests               | Выполнено |
| 77 | Панель EDITOR                          | 13; 22.7            | EditorDashboardPage.tsx;<br>Рисунок 8   | Выполнено |
| 78 | Управление документами редактором      | 13; 22.7            | /api/editor/documents; video            | Выполнено |
| 79 | Hide и restore документов              | 13; 22.7            | editor endpoints; video                 | Выполнено |
| 80 | Изменение метаданных и тегов           | 13; 22.7            | metadata endpoint; editor service       | Выполнено |
| 81 | Переиндексация отдельного документа    | 17; 22.7            | POST /api/editor/documents/{id}/reindex | Выполнено |
| 82 | Панель ADMIN                           | 13; 22.7            | AdminDashboardPage.tsx;<br>Рисунок 9    | Выполнено |
| 83 | Управление пользователями              | 15; 22.7            | /api/admin/users; screenshot 10         | Выполнено |
| 84 | Назначение ролей                       | 15; 22.7            | role endpoint; auth screenshot 07       | Выполнено |
| 85 | Блокировка/разблокировка пользователей | 15; 22.7            | block/unblock endpoints; video          | Выполнено |
| 86 | Управление источниками                 | 22.7                | /api/admin/sources; screenshot 11       | Выполнено |
| 87 | Запуск синхронизации                   | 6; 22.7             | POST /api/admin/sources/{id}/sync       | Выполнено |
| 88 | Управление jobs                        | 17; 22.7            | /api/admin/jobs; screenshot 12          | Выполнено |
| 89 | Retry/cancel заданий через UI          | 17; 22.7            | JobsPage + admin job endpoints          | Выполнено |
| 90 | Просмотр audit                         | 22.7                | /api/admin/audit; screenshot 13         | Выполнено |

| №   | Реализованный пункт                                               | Где описан в отчёте | Доказательство                        | Статус    |
|-----|-------------------------------------------------------------------|---------------------|---------------------------------------|-----------|
| 91  | Просмотр system status                                            | 18; 22.7            | /api/admin/system; screenshot 14      | Выполнено |
| 92  | Управление поисковыми индексами                                   | 11; 22.7            | /api/admin/indexes; IndexesPage       | Выполнено |
| 93  | Журналирование административных действий                          | 15; 22.7            | audit_events; audit service           | Выполнено |
| 94  | Удаление секретных данных из audit                                | 14; 22.7            | audit sanitizer tests/security report | Выполнено |
| 95  | Docker Compose                                                    | 18; 22.8            | compose.yaml; config validation       | Выполнено |
| 96  | Контейнеры PostgreSQL, Qdrant, Ollama, backend, frontend, workers | 5; 18; 22.8         | 8 compose services                    | Выполнено |
| 97  | Persistent volumes                                                | 18; 22.8            | 4 named volumes in compose            | Выполнено |
| 98  | Healthchecks                                                      | 18; 22.8            | Docker healthchecks; acceptance       | Выполнено |
| 99  | Nginx и единый origin                                             | 5; 18; 22.8         | frontend/nginx.conf                   | Выполнено |
| 100 | Автоматические backend-тесты                                      | 19; 22.9            | 188 tests PASS                        | Выполнено |
| 101 | Автоматические frontend-тесты                                     | 19; 22.9            | 103 tests PASS                        | Выполнено |
| 102 | Проверка coverage                                                 | 19; 22.9            | 81,125085%; coverage artifact         | Выполнено |
| 103 | Интеграционные тесты Qdrant                                       | 19; 22.9            | Qdrant integration test suite         | Выполнено |
| 104 | Acceptance testing                                                | 19; 22.9            | acceptance report PASS                | Выполнено |
| 105 | Проверка index consistency                                        | 11; 19; 22.9        | index consistency PASS                | Выполнено |
| 106 | Реальная работа поиска                                            | 19; 22.9            | live BM25/Vector/Hybrid PASS          | Выполнено |
| 107 | Реальная работа RAG                                               | 19; 22.9            | live RAG/citations PASS               | Выполнено |
| 108 | Реальные показатели корпуса                                       | 6; 20; 22.9         | corpus manifest: 10 700 real          | Выполнено |
| 109 | Реальные скриншоты интерфейса                                     | 13; 22.9            | 14 PNG в отчет/Скриншоты              | Выполнено |
| 110 | Инструкция запуска                                                | 18; 22.10           | README.md и Makefile                  | Выполнено |
| 111 | SQL-запросы с JOIN                                                | 9; 22.10            | docs/final/sql-demo.sql               | Выполнено |
| 112 | SQL-запросы с GROUP BY                                            | 9; 22.10            | docs/final/sql-demo.sql               | Выполнено |
| 113 | SQL-запросы с подзапросами                                        | 9; 22.10            | docs/final/sql-demo.sql               | Выполнено |
| 114 | SQL-запросы с CTE                                                 | 9; 22.10            | docs/final/sql-demo.sql               | Выполнено |
| 115 | SQL-запросы с оконными                                            | 9; 22.10            | docs/final/sql-demo.sql               | Выполнено |

| №   | Реализованный пункт                          | Где описан в отчёте | Доказательство                                                                   | Статус             |
|-----|----------------------------------------------|---------------------|----------------------------------------------------------------------------------|--------------------|
|     | функциями                                    |                     |                                                                                  |                    |
| 116 | Архив исходного кода                         | 22.10               | ZIP в текущем дереве отсутствует                                                 | Выполнено частично |
| 117 | Документация сервисов и скриптов             | 22.10               | отчет/Код_сервисов_и_скриптов_PyAnswer.pdf                                       | Выполнено          |
| 118 | Видео всех доступных сценариев               | 22.10               | отчет/PyAnswer_demo_full.webm, 05:15                                             | Выполнено          |
| 119 | Презентация проекта                          | 22.10               | отчет/PyAnswer_Презентация.pdf/.pptx                                             | Выполнено          |
| 120 | Описание индивидуального вклада автора       | 22.11               | docs/final/contribution.md; подраздел 22.11                                      | Выполнено          |
| 121 | Импорт официального Stack Exchange Data Dump | 6; 22.1             | dump.py, import_stackexchange_dump.py, unit-тесты; массовый импорт не запускался | Выполнено          |

## 23. Соответствие требованиям кейса «Поисковик, RAG»

**Таблица 19 — Итоговое соответствие требованиям кейса**

| Требование кейса                 | Реализация PyAnswer                                         | Результат                             |
|----------------------------------|-------------------------------------------------------------|---------------------------------------|
| Парсинг информационного ресурса  | Stack Exchange API client и restart-safe source sync worker | Выполнено; 10 700 реальных документов |
| Загрузка в поисковые индексы     | Indexer создаёт sparse/dense points из PostgreSQL chunks    | Выполнено; 25 214 points              |
| Сервер поисковых запросов        | FastAPI GET /api/search                                     | Выполнено; live Search PASS           |
| top-N документов                 | Limit/pagination и reranked result list                     | Выполнено                             |
| score                            | Fusion/final score в Search response и карточке             | Выполнено                             |
| Содержимое результатов           | Title, excerpt/highlights, tags и document page             | Выполнено                             |
| Ссылки на оригинальные источники | source_url и attribution на карточке/странице               | Выполнено                             |
| RAG                              | Local Ollama grounded generation                            | Выполнено; live RAG PASS              |
| Hybrid Search                    | Sparse BM25 + dense HNSW через weighted RRF                 | Выполнено                             |
| Reranker                         | Локальный Qwen3 Reranker service                            | Выполнено                             |
| Фильтрация                       | Source/tag/accepted/code/status filters                     | Выполнено                             |
| Пользовательский интерфейс       | React + TypeScript + Vite, production Nginx                 | Выполнено                             |
| Авторизация                      | Argon2id, opaque server sessions, HttpOnly, CSRF            | Выполнено                             |
| Три роли                         | USER, EDITOR, ADMIN и 20 permissions                        | Выполнено                             |
| Аудит                            | Sanitized audit_events и read-only admin UI                 | Выполнено                             |
| Фоновые обновления               | Durable jobs, checkpoint, lease, heartbeat, retry/cancel    | Выполнено                             |

Все обязательные требования кейса “Поисковик, RAG” выполнены. Дополнительно реализованы гибридный поиск, локальный reranker, потоковый RAG с проверяемыми цитатами, ролевая модель, аудит, фоновые задания и административный интерфейс.

## 24. Ограничения и направления развития

Корпус содержит 10 700 реальных документов и не достиг целевых 25 000; `initialSyncCompleted=false`. Причина — ограничение дневной `quota` Stack Exchange, а не отсутствие `pipeline`. Минимальное обязательное значение 5 000 превышено.

`Search/RAG datasets` присутствуют, но `reviewed qrels` не завершены, поэтому полноценные `quality metrics` не публикуются. `Browser Playwright E2E` отсутствует; критический путь проверен HTTP acceptance и реальными скриншотами. Локальные `model latency` и `throughput` зависят от CPU/GPU/RAM; единичные запросы не являются `benchmark`. В `multi-replica deployment` `process-local rate limiter` потребует общего `store`.

Направления развития: продолжение `incremental import` после восстановления `quota`; ручная разметка `qrels`; `browser E2E`; `latency profiling` и `batching`; централизованный `rate-limit store`; расширение `corpus` при сохранении `attribution/security model`.

## Заключение

Цель работы достигнута: создана локальная информационно-поисковая RAG-система PyAnswer по русскоязычной базе знаний Python. Реализованы ingestion и обработка данных, PostgreSQL source of truth, sparse/dense Qdrant index, BM25/vector/hybrid retrieval, локальный reranker, Ollama generation, RAG с SSE и цитатами, React UI, session authorization, RBAC, audit, durable jobs и Docker deployment.

Работоспособность подтверждается corpus/release manifests, 25 214 согласованными chunks/points, 188 backend и 103 frontend tests, coverage 81,125085%, index consistency PASS и final acceptance PASS. Ограничения зафиксированы честно и не отменяют выполнение обязательных требований кейса.

## Список источников

1. Репозиторий PyAnswer: <https://github.com/4iriks/praktika>, commit 60cc0680cba0934d2359701778e743f51f397a8d.
2. README, `compose.yaml`, Makefile и документация проекта PyAnswer.
3. Stack Exchange API v2.3: <https://api.stackexchange.com/docs>.
4. PostgreSQL 16 Documentation: <https://www.postgresql.org/docs/16/>.
5. Qdrant Documentation: <https://qdrant.tech/documentation/>.
6. FastAPI Documentation: <https://fastapi.tiangolo.com/>.
7. React Documentation: <https://react.dev/>.
8. Ollama Documentation: <https://github.com/ollama/ollama>.
9. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond.
10. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

## Приложение А. Полная ER-диаграмма

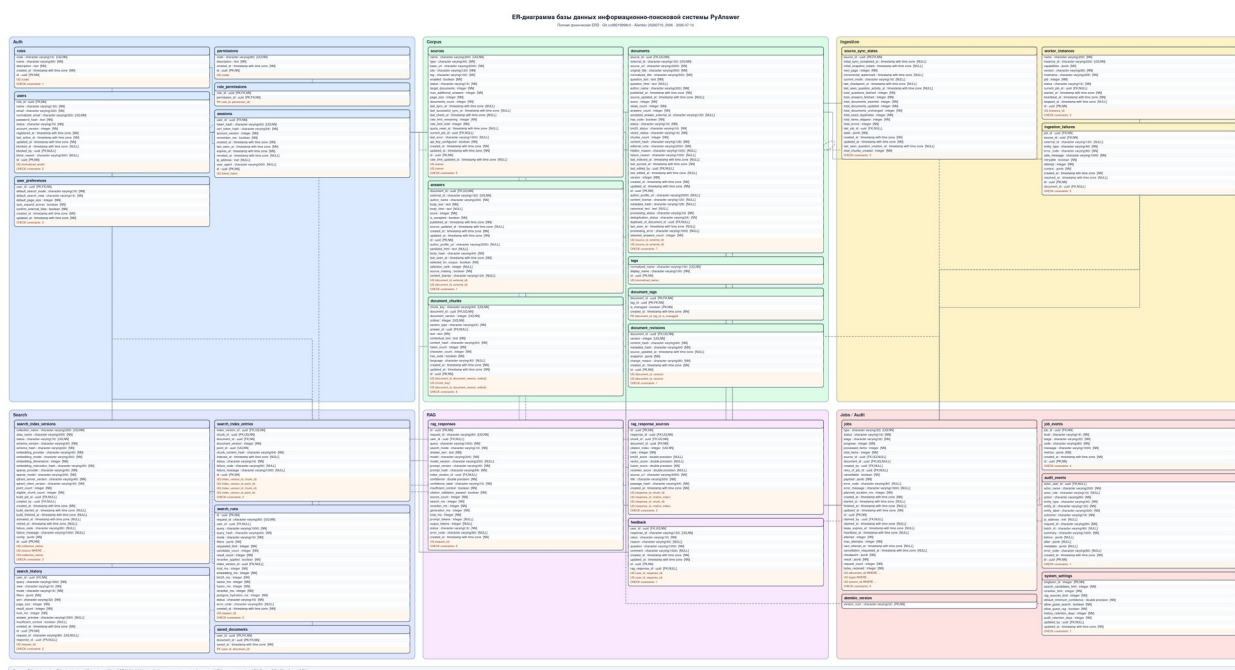


Рисунок А.1 — Полная физическая ER-диаграмма PostgreSQL PyAnswer

## **Приложение Б. Полный SQL-файл**

Read-only набор из 20 запросов расположен в docs/final/sql-demo.sql. Он включает JOIN, GROUP BY, FILTER, NOT EXISTS, CTE, row\_number, dense\_rank и aggregate subqueries.

## Приложение В. Карта исходного кода

| Область           | Фактический путь                                    |
|-------------------|-----------------------------------------------------|
| Frontend          | frontend/src/                                       |
| FastAPI routes    | backend/app/api/routes/                             |
| Schemas/services  | backend/app/schemas/, backend/app/services/         |
| SQLAlchemy/models | backend/app/db/models/                              |
| Stack Exchange    | backend/app/integrations/stackexchange/             |
| Processing        | backend/app/processing/                             |
| Qdrant/Ollama     | backend/app/integrations/qdrant/, embeddings/, llm/ |
| Workers           | backend/app/workers/                                |
| Alembic           | backend/alembic/versions/                           |
| Tests             | backend/tests/, frontend/src/*.test.*               |
| Operations        | scripts/, Makefile, compose.yaml                    |

## **Приложение Г. Материалы подтверждения**

- artifacts/corpus-manifest.json;
- artifacts/index-consistency-report.json;
- artifacts/acceptance-report.json;
- artifacts/release-manifest.json;
- artifacts/backend-coverage.json;
- docs/final/presentation/Скриншоты/;
- docs/final/auth-and-roles/;
- отчет/ER/.